Log-Structured Merge Trees

(LSM Trees)

Not In the TextBook

Instructor: Barzan Mozafari

University of Michigan, Ann Arbor

Why Do We Need LSM Trees?

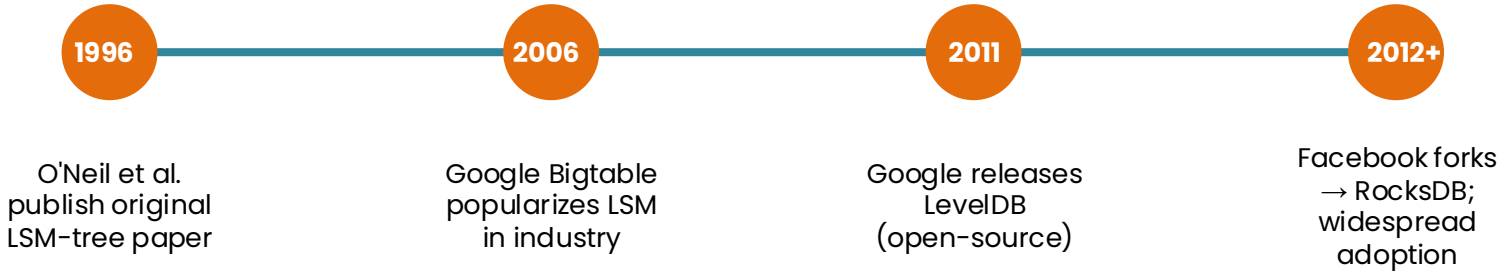
The B+ Tree Bottleneck

- Every insert requires random disk I/O: read page, modify, write back
- Maintaining a B+ tree index can double the I/O cost of a transaction
- Terrible for write-heavy workloads: logging, IoT, social feeds, time-series

The LSM Tree Insight

- Buffer all writes in memory (fast!)
- Flush sorted batches to disk using sequential I/O
- Sequential I/O is 10-100× faster than random I/O
- Trade-off: reads are harder (data spread across multiple sorted files)

History and Adoption



Systems using LSM Trees today:

Embedded Engines: LevelDB, RocksDB, Pebble

Distributed DBs: Cassandra, HBase, ScyllaDB

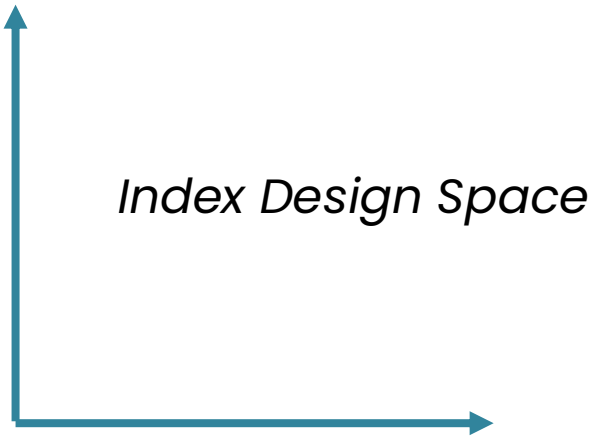
SQL DBs (via LSM engine): CockroachDB, TiDB (TiKV), YugabyteDB, MyRocks (MySQL+RocksDB)

Search / Time-Series: InfluxDB (TSM: an LSM-inspired time-series storage engine)

LSM Trees as an Index Structure

Organization Structure for k*

- Hash-based
  (+) Equality search
- Tree-based
  (+) Range, equality search
  - B+Tree (dynamic)
  - LSM Tree (dynamic)
  - ISAM (static)



Data Entry (k*) Contents

1. Actual Data record index = file
   - Most common for LSMs: Key-value pairs live directly in the sorted runs.
2. <k, rid>
   - actual records in a diff heap file file
3. <k, list of rids>: actual records in a diff file
   - Rarely used for LSMs

LSM Tree Architecture

What if there's a crash?
LSMs use WAL (covered later)

In-Memory Buffer (MemTable)

All writes go here first
Capacity: P entries

Sorted in-memory structure

flush as SSTable →

On Disk

Level 1 ($P \cdot T$ entries)

Level 2 ($P \cdot T^2$ entries)

⋮

Level L ($P \cdot T^L \approx N$)

Key Terminology:

SSTable (Sorted String Table) | 1 | 3 | 5 | 8 | = a sorted, immutable file on disk

Sorted Run | 1 | 3 | 5 | 8 | | 10 | 12 | 15 | | 20 | 25 | = one or more SSTables with non-overlapping key ranges

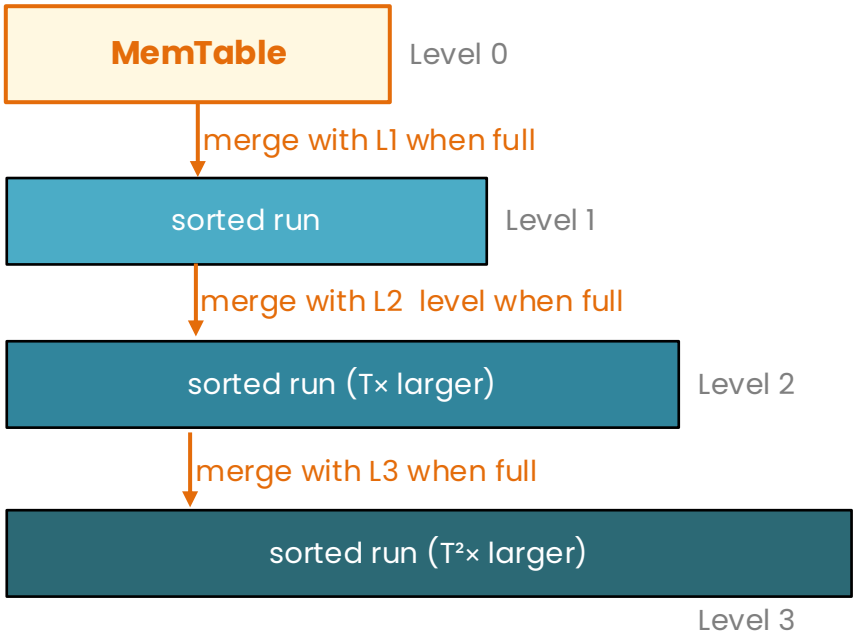
N = total entries B = entries/page P = buffer capacity
T = size ratio between levels L = disk levels ≈ $\log_T(N/P)$

Fun Fact: Why LSMs are called "trees"? *The original 1996 paper used B-trees for each component. Modern LSM systems use sorted arrays/runs instead, but the name stuck. Can also think of level hierarchy as a tree-like structure where data flows from the root (memory) down through levels.*

Two LSM Designs: Leveled vs Tiered

Leveled (Eager Compaction)

1 sorted run per level



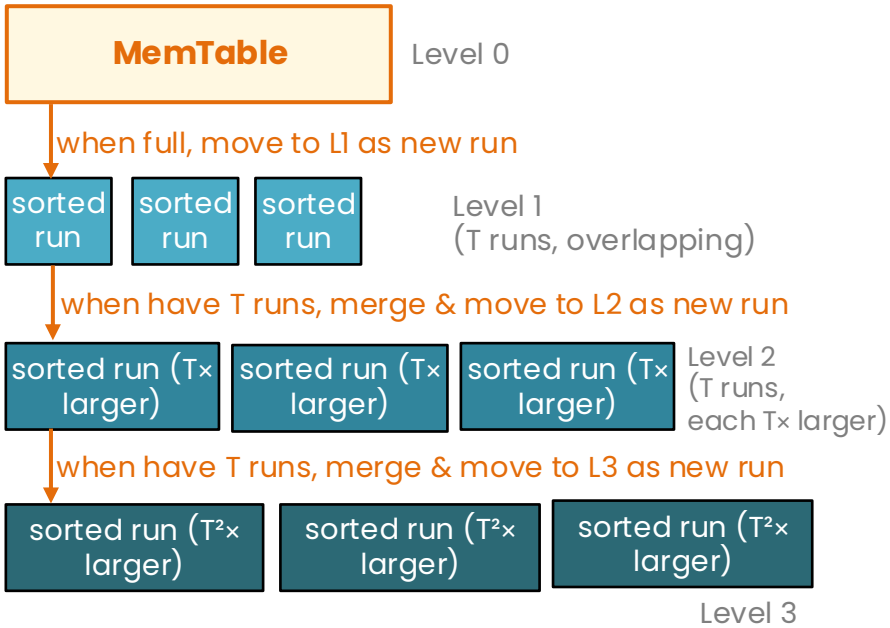
✓ **Read-optimized (1 run to search per level)**

✗ **High write amplification: O(T·L)**

✓ **Low space amplification: O(1/T)**

Tiered (Lazy Compaction)

Up to T runs per level (may overlap)



✓ **Write-optimized (no merge until T runs)**

✗ **Higher read cost: O(T·L) runs to search**

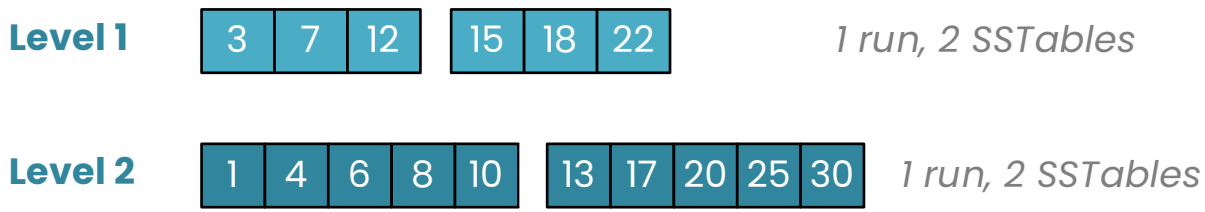
✗ **High space amplification: O(T)**

Both share: MemTable & flushing to disk levels.

Difference: max runs per level and when to merge.

Leveled Design (Eager Compaction)

Each level has exactly **one sorted run** (which may consist of multiple SSTables with non-overlapping key ranges).



Compaction:

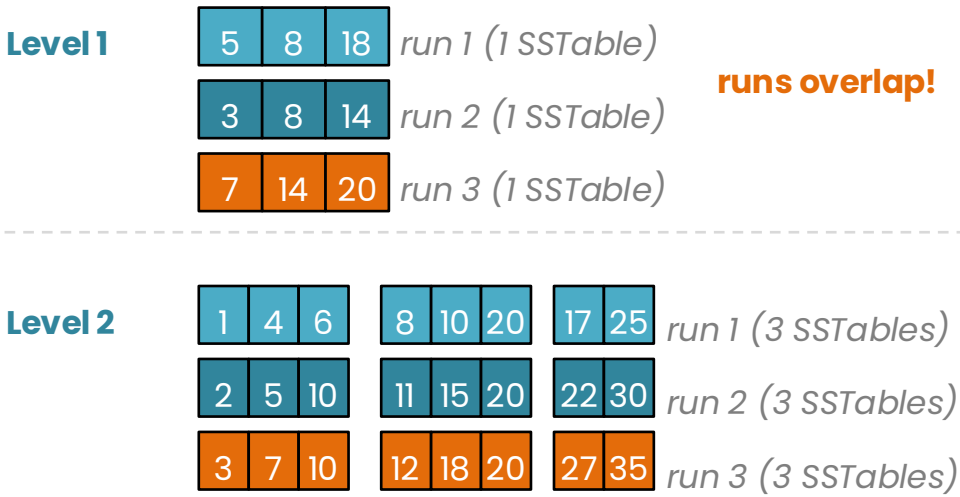
- when Level i overflows ($> P \cdot T^i$ entries), merge its run with Level i+1's run → new single run at Level i+1
- If level i+1 overflows, keep merging with next levels, recursively

Leveled Design is used by: LevelDB, RocksDB (default)

Tiered Design (Lazy Compaction)

Each level capacity: $P \cdot T^i$ entries, spread across **up to T sorted runs**.

SSTables within a run are non-overlapping. SSTables across runs **may overlap**.



Compaction: when Level i accumulates T runs → merge them all into one run and add it as a new new run to Level i+1.

Tiered Design is used by: Cassandra (size-tiered), HBase

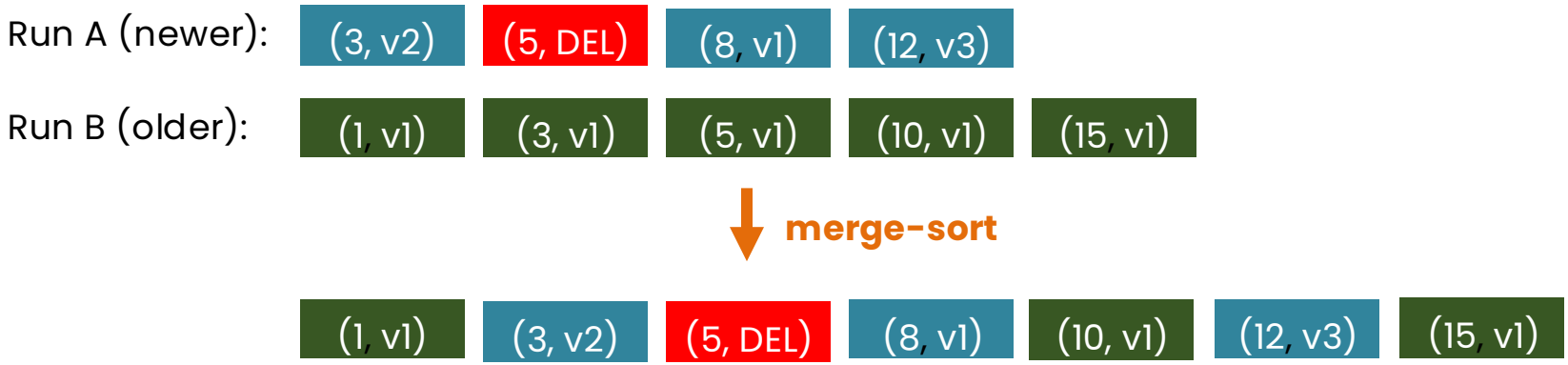
How Compaction (Merge) Works

This is how LSMs garbage-collect stale data

During a merge:

1. Keys remain sorted in the new run
2. Old versions are purged, i.e., only keep most recent version of each key
3. Tombstones also purged **when it's safe**: no older runs or lower levels exist

Example: consider 2wo runs with overlapping keys:



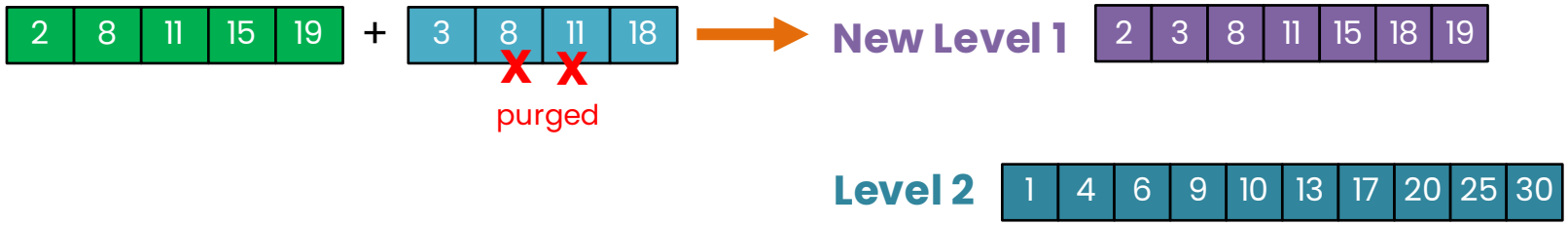
After: single merged run (old versions purged)

- **key 3:** kept v2 (newer), discarded v1 (older)
- **key 5:** tombstone met v1 → v1 removed. If we know no more versions of 5 exist we can also delete tombstone (i.e., delete is complete)

Leveled: Insert (1/3) — Flush & Merge

Step 1: Write to MemTable. When full, flush to Level 1.

Step 2: Level 1 already has a run → must merge immediately:



Inserts, updates, and deletes all follow the same path:

write to MemTable → when full, flush → merge with existing run in L1

Leveled: Insert (2/3) — Cascading

What if Level 1 now exceeds its capacity?

Cascading compaction: one flush can trigger merges at L1 → L2 → L3 → … all the way down!

Write Amplification:

Total data written to disk ÷ data originally written by user. If you insert 1 MB but the system writes 60 MB (due to repeated merges), write amplification = 60×.

Write amplification for Leveled Design: **O(T · L)**.

At each level transition, a record merges into a level that is T x larger, so it gets rewritten ~T times per transition. There are L transitions total. So: T rewrites per level, L levels: ***O(T · L)****.*

Leveled: Point Lookup

1 run per level, search 1 run per level.

Stop at first match. Worst case (not found): check all L levels.

Example: Find key **k = 15**, search top-down:

MemTable: [2, 8, 11] ① **not found**

Level 1: [3, 7, 12, 15, 18] ② **found!**

Level 2: [1, 4, 6, 9, 10, 13, 17, 20, 25, 30] ③ **skip**

Look up cost: O(L · log(N/B)) I/Os (L levels × binary search per level)

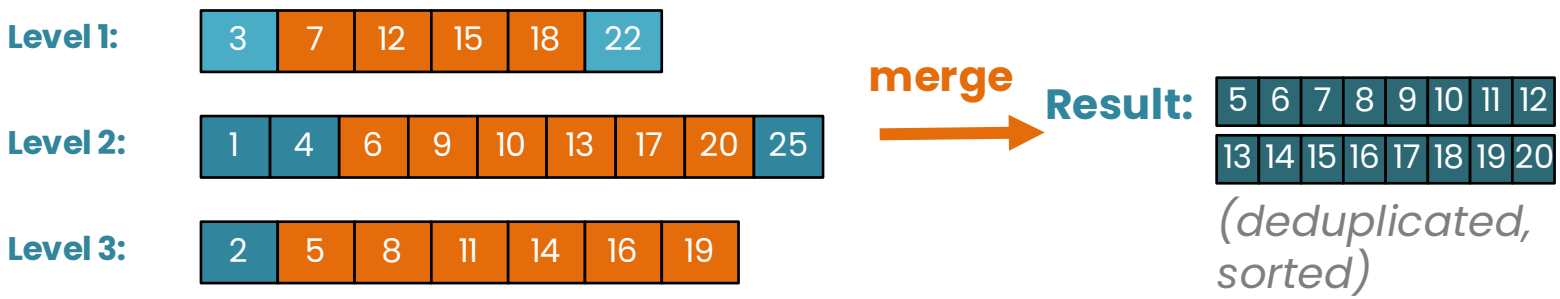
Binary search at level i is over $P \cdot T^i/B$ pages, thus exact cost is $\Sigma_i \log(P \cdot T^i/B) = L \cdot \log(P/B) + \log(T) \cdot L(L+1)/2$. Since $N \approx P \cdot T^L$, this is $O(L \cdot \log(N/B))$. This is asymptotically correct but overstates the cost at every level except the last, because it uses the largest run's size as a bound for all levels.

Leveled: Range Query

How it works:

1. Binary search each run to find start of range [5, 20]
2. Scan forward collecting matching entries (orange cells)
3. Merge the L streams, keeping newest version of each key
4. Return merged, sorted result

Example: Find all keys in [5, 20]: must check ALL levels:



Range Query Cost: $O(L \cdot \log(N/B) + s/B)$

Binary search each of L levels to find the range start: log(N/B) I/Os per level, in total L·log(N/B). Then scan all s matching entries: s/B I/Os, since scan cost depends only on result size, not L; merge is CPU-only. Total: **O(L·log(N/B) + s/B)**.

Quiz: Leveled Design

Consider a leveled LSM tree with T = 10 and the following state:

MemTable: [45, 52, 67, 81, 93]

Level 1:

10	25	38	42	55	60	73	88

Level 2:

3	8	12	18	22	30	35	40	48	52	58	65	70	78	85	90	95

Questions:

Q1: To look up key 55, which runs are checked? In what order?

Q2: To look up key 99 (doesn't exist), how many runs total are checked?

Q3: Key 52 is updated. Where does the new value go? Which version is returned on lookup?

Q4: For range query [40, 70], which levels must be scanned? Can we skip any?

Answer

Consider a leveled LSM tree with P=7, T = 10 and the following state:

MemTable: [45, 52, 67, 81, 93]

Level 1:	10	25	38	42	55	60	73	88

Level 2:	3	8	12	18	22	30	35	40	48	52	58	65	70	78	85	90	95

Questions:

Q1: To look up key 55, which runs are checked? In what order?

- MemTable (not found) → Level 1 (FOUND, stop). Level 2 never checked.

Q2: To look up key 99 (doesn't exist), how many runs total are checked?

- All 3 (MemTable + L1 + L2). Worst case for leveled design.

Q3: Key 52 is updated. Where does the new value go? Which version is returned on lookup?

- New ⟨52, v_new⟩ written to MemTable (in-place update since MemTable is mutable). Old copy persists at Level 2 until compaction. MemTable version returned on lookup.

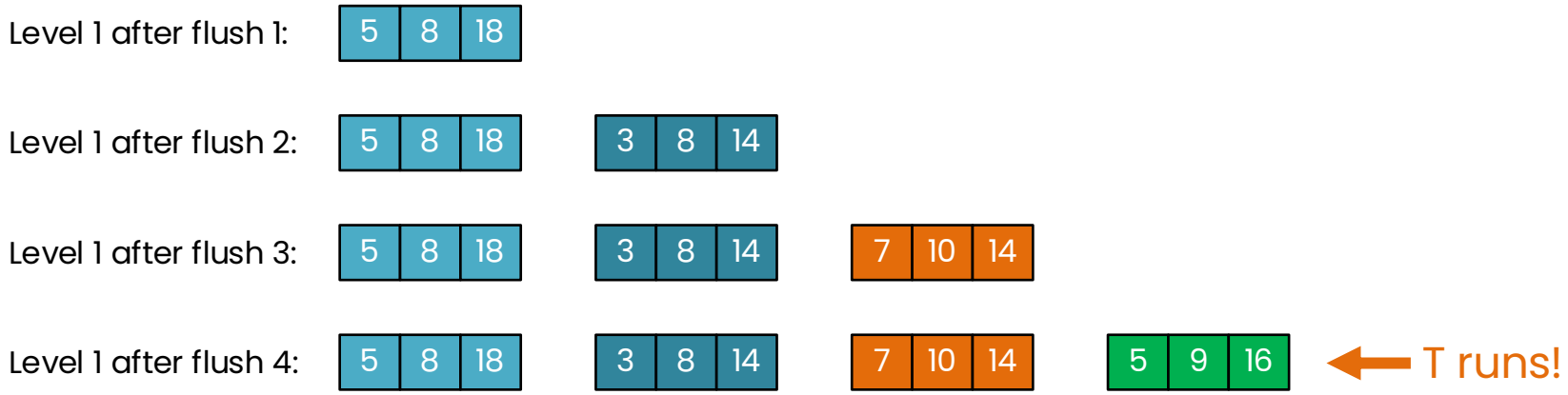
Q4: For range query [40, 70], which levels must be scanned? Can we skip any?

- ALL levels must be scanned (range queries can never stop early).

Tiered: Insert — Flush MemTable When Full

Add to MemTable. If full, flush to Level 1 as a new run. **No merge!**

Example: T = 4



Writes are cheap: no merging until T runs accumulate!

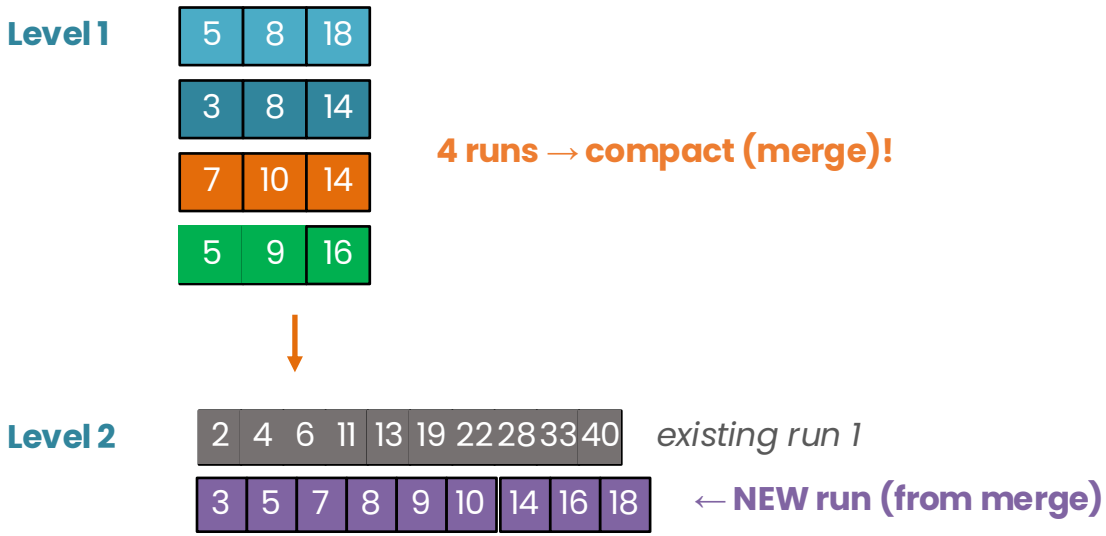
Space Amplification:

Physical bytes stored ÷ logical live (unique) bytes. Perfect = 1× (no redundancy).

With tiered, same key can exist in multiple runs → up to O(T) amplification.

Tiered: Compaction Trigger

When T runs accumulate → merge all into one new run at Level i+1



Level 1 is now empty. Level 2 now has 2 runs (up to T allowed).

Write amplification: O(L)

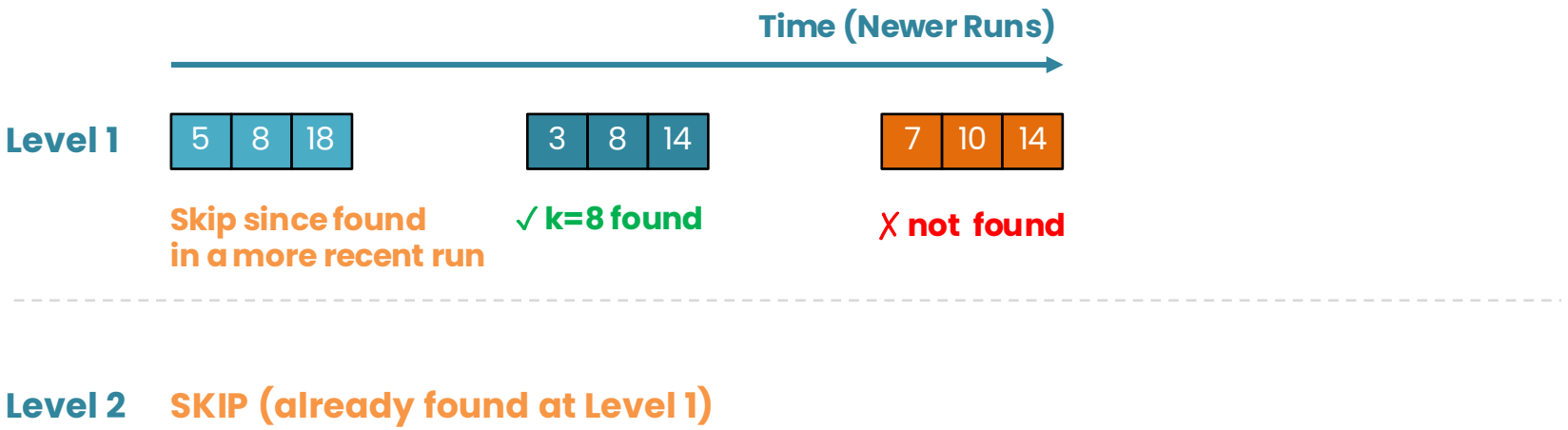
Each entry merged once per level. That's T× cheaper than leveled!

Tiered: Point Lookup

How it works:

1. Search top-down. If found at level L, skip L+1, …
2. Within each level, search the most recent runs first. If found, skip the rest.

Example: find k = 8



Look up cost: **$O(T \cdot L \cdot \log(N/B))$ I/Os** worst case (not found)

In the worst case, binary search all T runs across L levels. Each run requires a binary search costing log(N/B) I/Os to locate the candidate page.

Tiered: Range Query

Must scan **ALL runs at ALL levels** (can't stop early)

Example: Find keys in [5, 20]

Level 1

scan all 3!

Level 2 … also scan all T runs here

Level L … and all T runs at every level

Range query cost: **$O(T \cdot L \cdot \log(N/B) + T \cdot s / B)$ I/Os**

Must scan all T·L runs — no early exit since runs overlap and any run could contain range keys. Binary searching each to find the range start costs T·L·log(N/B). Scan cost is T·s/B not s/B because overlapping runs mean you read up to T copies of the s results before deduplication.

Quiz: Tiered Design

Consider a tiered LSM tree with T = 3 and the following state:

MemTable: [20, 35, 50]

Level 1:

5	15	25	40	*run 1 (older)*
10	25	30	45	*run 2 (newer)*

Level 2:

2	7	12	18	22	28	33	38	42	48	55	60	*run 1*

Questions:

Q1: Look up key 25: which runs are checked? Which of the values in the L1 runs are returned?

Q2: Look up key 60: how many runs total must be checked?

Q3: If we delete key 25: where does the tombstone go? What about stale copies of 25?

Q4: If we delete key 20: where does the tombstone go?

Answer

Consider a tiered LSM tree with P=5 and T = 3 and the following state:

MemTable: [20, 35, 50]

Level 1:

5	15	25	40	*run 1 (older)*
10	25	30	45	*run 2 (newer)*

Level 2:

2	7	12	18	22	28	33	38	42	48	55	60	*run 1*

Questions:

Q1: Look up key 25: which runs are checked? Which of the values in the L1 runs are returned?

- MemTable (not found) → check run 2 in L1. Since already found, Skip run 1 since it's older. Skip Level 2 too.

Q2: Look up key 60: how many runs total must be checked?

- MemTable (not found) → all L1 runs (not found) → L2 (found). Total: 4 checked (MemTable + 2 L1 runs + 1 L2 run).

Q3: If we delete key 25: where does the tombstone go? What about stale copies of 25?

- Tombstone written to MemTable. 2 stale copies on disk (two runs in L1) will only get purged in subsequent merges.

Q4: If we delete key 20: where does the tombstone go?

- Tombstone written to MemTable (will replace current version of 20 since MemTable is mutable).

Cost Summary (Before Optimizations)

Can we do better?
Yes (Fence Pointers + Bloom Filters)

Metric	LSM (Leveled)	LSM (Tiered)	B+ Tree
Insert	$O(T \cdot L / B)$	$O(L / B)$	$O(1)$ or $O(h)$ rand.
Point Lookup	$O(L \cdot \log(N/B))$	$O(T \cdot L \cdot \log(N/B))$	$O(1)$ or $O(h)$ rand.
Range Query	$O(L \cdot \log(N/B)+s/B)$	$O(T \cdot L \cdot \log(N/B)+T \cdot s/B)$	$O(h+s/B)$ or $O(1+s/B)$ rand.
Write Amp	$O(T \cdot L)$	$O(L)$	$O(1)$
Space Amp	$O(1+1/T) \approx 1\times$	$O(T)$	$\approx 1.5\times$ (67% fill factor)

Note: traversing a B+tree is **random** I/O while LSM I/Os are mostly **sequential**!

**Space Amp (Leveled):* *In steady state, level L holds a (T-1)/T fraction of all unique data, so all smaller levels add ~1/T redundancy → total ≈ 1 + 1/T. This excludes temporary compaction buffers, which can transiently up to double the space at the largest level during an active merge.*

Problem: Too much disk I/O

Example:

To find **k=17** in a run, we have to do a **binary search ON DISK**

Sorted Run (4 pages on disk):

Page 1 [1–8]	Page 2 [9–15]	Page 3 [16–24]	Page 4 [25–35]

Binary Search

1. Read Page 2 [9–15] from disk → 17 > 15 → go right (1 I/O)

2. Read Page 3 [16–24] from disk → found! (1 I/O)

Total: O(log(pages)) = O(log(N/B)) disk I/Os per run

Can we do less I/Os?

Optimization 1: Fence Pointers

Idea: Keep a small in-memory array: [min, max] key of every disk page.

Example: Look up k = 17

IN MEMORY (free!)

Binary search fence pointers:

{Pg1: [1,8], Pg2: [9,15], Pg3: [16,24], Pg4: [25,35]}

→ **17 ∈ [16,24] → Page 3**

0 disk I/Os

ON DISK

Read exactly 1 page:

Pg 1 [1, 3, 5, 8]	Pg 2 [9, 12, 14,15]	Pg 3 [16, 20, 21, 24]	Pg 4 [25, 30, 31,35]

→ **read Page 3, find key 17**

1 disk I/O!

1. Eliminates log factor:

- **Before:** O(log(N/B)) I/Os per run → **After:** O(1) I/O per run
- **Leveled: O(L·log(N/B))→O(L)**
- **Tiered: O(T·L·log(N/B))→O(T·L)**

2. Requires little memory (~0.4% of data) → All modern LSMs use fence pointers

Optimization 2: Bloom Filters

Bloom Filter:

- A data structure for **membership test** (not an index!)
- Structure: **bit array of m bits** + **k hash functions**
- Each hash function maps key → position in [0, m-1]

How it works: assume k=3, m=16:

Insert "cat": h_1=2, h_2=7, h_3=13 → set bits 2,7,13 to 1

0	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0

Insert "dog": h_1=1, h_2=10, h_3=2 → set bits 1,10,2 to 1

0	1	1	0	0	0	0	1	0	0	1	0	0	1	0	0

Lookup "cat": h_1=2✓ h_2=7 ✓ h_3=13 ✓ → all 1 → **POSSIBLY in set** (true)
Lookup "bird": h_1=3 ✓ h_2=7 ✓ h_3=11✗ → bit 11=0 → **DEFINITELY NOT**
Lookup "mouse": h_1=1✓ h_2=2 ✓ h_3=7 ✓ → all 1 → **POSSIBLY in set** (false positive!)

- No false negatives, but false positives are possible
- Insertion and lookups supported; deletion and updates NOT supported

Bloom Filter Sizing

False positive rate: $\varepsilon \approx (1 - e^{(-kn/m)})^k$

- n is number of distinct keys will be inserted in the set (**m/n is "bits-per-key"**)
- It can be shown that **optimal k** = $(m/n) \cdot \ln(2)$
- Using optimal k, false positive rate simplifies to: $\varepsilon \approx (0.6185)^{(m/n)}$

Numerical examples (assuming optimal k):

Bits/key (m/n)	Optimal k	False Positive Rate ε	BF Size per 1M keys
5	3	~10%	0.6 MB
8	6	~2%	1.0 MB
10	**7**	**~1%**	**1.25 MB**
15	10	~0.1%	1.9 MB
20	14	~0.01%	2.5 MB

*Most systems use **10 bits/key**; good balance of accuracy vs memory.*

More bits → lower ε → fewer wasted I/Os, but more memory. Diminishing returns beyond ~15 bits/key.

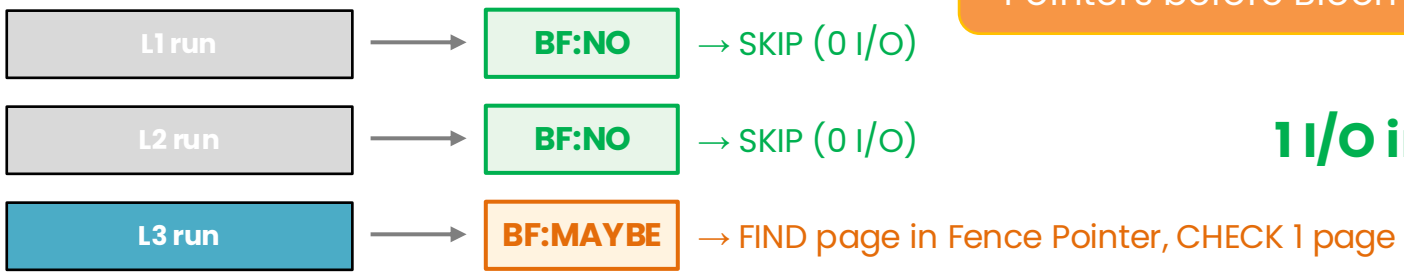
Using Bloom Filters to Reduce LSM's Disk I/Os for Point Lookups

Build a BF **per each run**, then you can ask "Is key k possibly here?"

- Maybe: use fence pointers for pages in that run to find the page (if any) and read the page to see if the key actually exists.
- No: they key is definitely not in that run (no need to look at fence pointers)

Q: Why don' t we look at Fence Pointers before Bloom Filter?

Example: point lookup for k = 42



1 I/O instead of 3!

Helps: Point Lookups

Expected wasted I/Os = (# runs) × ε

Leveled: $O(L \cdot \varepsilon)$, Tiered: $O(T \cdot L \cdot \varepsilon)$

With ε ≈ 1% and typical L (5–7 levels), this is close to 1 I/O in practice

Does NOT Help: Range Queries

BF tests ONE key. Can't test a range

Must still check every run for ranges

Updated Cost Summary (with Fence + Bloom Filter opts)

N = total entries, B = entries/page, P = buffer capacity, T = size ratio between levels, L = disk levels ≈ $\log_T(N/P)$, s=result size for range queries, ε=Bloom Filter's False Positive rate, h = height of B+ tree

Metric	LSM (Leveled)	LSM (Tiered)	B+ Tree
Insert	$O(T \cdot L / B)$	$O(L / B)$	O(1) or O(h) rand.
Point Lookup*	$O(L \cdot \varepsilon)$	$O(T \cdot L \cdot \varepsilon)$	O(1) or O(h) rand.
Range Query	$O(L + s/B)$	$O(T \cdot L + T \cdot s/B)$	O(h+s/B) or O(1+s/B) rand.
Write Amp	$O(T \cdot L)$	$O(L)$	O(1)
Space Amp**	$O(1+1/T) \approx 1\times$	$O(T)$	≈1.5× (67% fill factor)

Traversing a B+tree's height is **random** I/O while LSM I/Os are mostly **sequential** (next slide)

Deciding LSM design:

Write-heavy + point lookups → **Tiered**

Read/range-heavy → **Leveled** (RocksDB default)

***Point lookup:** With $\varepsilon \approx 1\%$ and typical L (5–7), expected I/O is close to 1 in practice, but asymptotic still depends on L.*

****Space Amp (Leveled):** Steady-state, excluding temporary compaction buffers.*

Numerical Example

Assume:

- $N=10^8$, $B=100$, $T=10$, $L=4$, $\varepsilon=1\%$, 1 rand I/O=10ms, 1 seq I/O=0.1ms
- All index nodes of B+ tree in memory; leaves on disk

Insertion cost per entry:

- B+ tree = 1 rand I/O = 10ms
- Leveled LSM= $T \cdot L/B$ = 0.4 seq I/O = 0.04ms (250× faster)
- Tiered LSM = L/B = 0.04 seq I/O = 0.004ms (2500× faster);

Point lookup cost per entry:

- B+ tree = 1 rand = 10ms
- Leveled LSM = $(1+L \cdot \varepsilon)$ rand I/O = 1.04 rand I/O ≈ 10ms
- Tiered LSM = $(1+T \cdot L \cdot \varepsilon)$ rand I/O = 1.4 rand I/O ≈ 14ms.

Summary

- LSM trees: buffer in memory → flush sorted SSTables to disk (sequential I/O)
- SSTable = sorted, immutable file. Sorted run = 1+ non-overlapping SSTables
- Leveled: 1 run/level, read-optimized, write amp O(T·L)
- Tiered: up to T runs/level (may overlap), write-optimized, write amp O(L)
- Merge purges old versions (garbage collection)
- Fence pointers: O(log) → O(1) per run.
- Bloom filters: point lookups ≈ O(1)
- Choose leveled vs tiered based on workload: writes vs range queries